DIGITAL SYSTEMS ELECTRONICS

Academic year 2024-2025

LABORATORY NR. 2

DUE DATE: March 24, 2025
DELIVERY DATE: March 22, 2025

GROUP 20

CONTRIBUTIONS:

- Erfan Taheri
- Raman Jahandari
- Bardia Vafadar
- Ali Mansouri

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

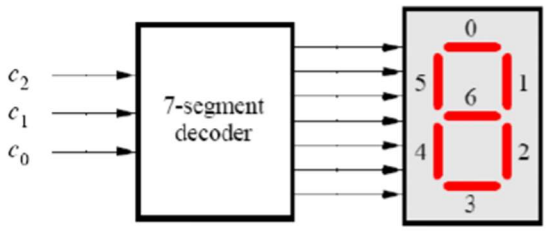
Introduction

This lab focuses on designing and implementing digital circuits using switches, decoders, and seven-segment displays on the DE1-SoC board. The objective is to control a 7-segment display via a decoder, implement multiplexing for dynamic display control, and develop binary-to-decimal and binary-to-BCD conversion circuits. VHDL modeling, functional validation in ModelSim, and FPGA implementation in Quartus Prime will be performed, followed by timing simulations to analyze propagation delays.

1. Controlling a 7-segment display

The goal of this part is to display the individual letters of the word **"HELO" (with one L)** sequentially on five of the six 7-segment displays on the board. To achieve this, we designed an entity called **"decoder7"**, which takes three switches as inputs and controls the display output through the vector **"HEX0"**. This output vector has seven bits corresponding to each segment and if the bit is 1, it means that the segment is OFF, and if the bit is 0, it means that the segment should be ON.

The 3-bit inputs are assigned to one of the preassigned characters, the four letters of H, E, L and O, and blank mode which means all the segments of the display set to OFF (all 1's). The following table shows the corresponding input values to each letter.



$c_2c_1c_0$	Character
000	H
001	E
010	L
011	O
100	
101	
110	
111	

Figure 1. circuit symbol of the decoder and the 7-segment display, and the corresponding character codes

The circuit described in the file 'decoder7.vhd' includes an architecture that implements a 7-segment decoder, mapping a 3-bit input (C) to a 7-bit output (HEX0),controlling a 7-segment display to show characters 'H', 'E', 'L', 'O', or the blank. The decoder uses conditional statements to assign the correct segment patterns, with a default blank state ("1111111") which also minimize undefined errors during simulation (#2 Golden rule of combinational circuits, to have the outputs assigned for any possible combination of input signals).

```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.ALL;
3
4   ENTITY decoder7 IS
5       PORT (
6           C : IN STD_LOGIC_VECTOR(0 to 2);  -- 3-bit input
7           HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6)  -- 7-bit output for 7-segment display
8       );
9   END decoder7;
10
11  ARCHITECTURE Behavior OF decoder7 IS
12  BEGIN
13      PROCESS (C)
14      BEGIN
15          HEX0 <= "1111111"; -- Reducing undefined errors in the Time simulation
16          IF C = "000" THEN HEX0 <= "1001000"; -- Display 'H'
17          ELSIF C = "001" THEN HEX0 <= "0110000"; -- Display 'E'
18          ELSIF C = "010" THEN HEX0 <= "1110001"; -- Display 'L'
19          ELSIF C = "011" THEN HEX0 <= "0000001"; -- Display 'O'
20          ELSE HEX0 <= "1111111"; -- Default blank
21          END IF;
22      END PROCESS;
23  END Behavior;
24
```

Figure 2. showing the decoder7 code

To test the design, we used a testbench named **decoder7_tb**. Initially, we added to the testbench the 7-segment decoder as a component, and then we mapped the inputs and outputs. The testbench assigns a new input value to the 3-bit input C every 10 ns.

```
 2  USE ieee.std_logic_1164.ALL;
 3  USE ieee.std_logic_textio.ALL;
 4  USE std.textio.ALL;
 5
 6  ENTITY decoder7_tb IS
 7  END decoder7_tb;
 8
 9  ARCHITECTURE testbench OF decoder7_tb IS
10      SIGNAL C : STD_LOGIC_VECTOR(2 DOWNTO 0);
11      SIGNAL HEX0 : STD_LOGIC_VECTOR(0 TO 6);
12
13      COMPONENT decoder7
14          PORT (
15              C : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
16              HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6)
17          );
18      END COMPONENT;
19
20  BEGIN
21      UUT: decoder7 PORT MAP (C, HEX0);
22
23      PROCESS
24      BEGIN
25          -- Test all input combinations
26          C <= "000"; WAIT FOR 10 ns;
27          C <= "001"; WAIT FOR 10 ns;
28          C <= "010"; WAIT FOR 10 ns;
29          C <= "011"; WAIT FOR 10 ns;
30          C <= "100"; WAIT FOR 10 ns;
31          C <= "101"; WAIT FOR 10 ns;
32          C <= "110"; WAIT FOR 10 ns;
33          C <= "111"; WAIT FOR 10 ns;
34
35          -- End simulation
36          WAIT;
37      END PROCESS;
38  END testbench;
```

Figure 3. the decoder test-bench code.

The design was validated using ModelSim and tested on the DE1-SoC board, confirming correct functionality by toggling switches and observing the display output. The simulation with ModelSim was timing simulation, meaning that the changes was carried out with a few nano second delays, plus having for a short time undefined values for the signals at the beginning, which is depicted as red lines in the waveform results shown in Figure 4.

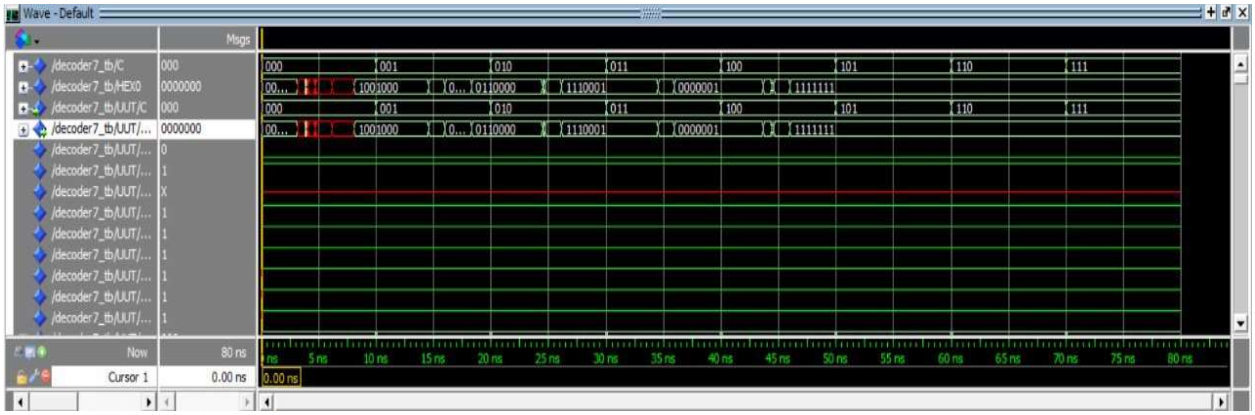
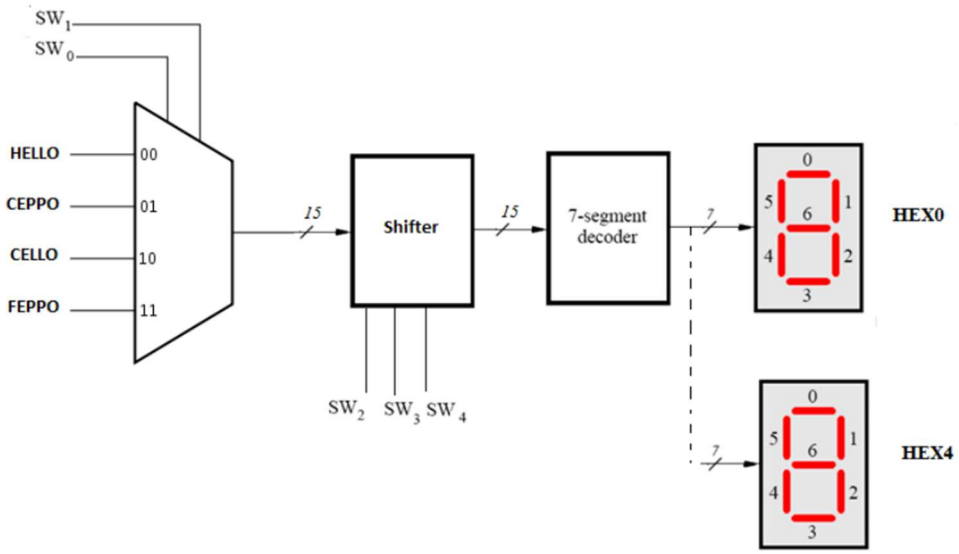


Figure 4. resulting waveform 7-bit outputs corresponding to the inputs, given in the test-bench.

2. Multiplexing the 7-segment display output



The second part builds upon the display driver developed in the previous implementation. All relevant **.vhd** files are organized within the subfolder. This circuit is designed to display one of four predefined words, which are hardcoded as input signals in a **4-to-1 multiplexer**. The selection of the desired word is controlled using the first two switches, **SW1-0**, according to the following truth table. The output of the multiplexer is 15-bit vector having the five 3-bits corresponding to each letter all together in one vector, which later goes to the shifter and then to the decoder. Below is the table corresponding to the multiplexer functionality :

SW0	SW1	m
0	0	HELLO
0	1	CEPPO
1	0	CELLO
1	1	FEPPO

Table 2: Multiplexer selection for different words.

Additionally, a **shifter** module allows the selected word to be shifted left by a specified number of positions, determined by switches **SW4-2**. The final output is displayed across five 7-segment displays, presenting the letters **H, E, L, O, P, F**, or a blank space in the correct sequence.

SW4	SW3	SW2	Character pattern
0	0	0	HELLO
0	0	1	ELLOH
0	1	0	LLOHE
0	1	1	LOHEL
1	0	0	OHELL

Table 3: Rotating the word HELLO on five 7-segment displays.

The key modules in this design are: **"mux.vhd"** for word selection, **"shifter.vhd"** for shifting operations, **"decoder7.vhd"** for 7-segment decoding, and **"part2.vhd"**, which serves as the main entity integrating all components. Below are some relevant code snippets from the three primary modules.

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY mux IS
PORT ( sel: IN STD_LOGIC_VECTOR(1 downto 0);
output : OUT STD_LOGIC_VECTOR(14 downto 0));
END mux;
ARCHITECTURE Behavior OF mux IS
BEGIN
    PROCESS (sel)
    BEGIN
        CASE sel IS
            WHEN "00" => output <= "000001010010011"; -- HELLO
            WHEN "01" => output <= "100001101101011"; -- CEPPO
            WHEN "10" => output <= "100001010010011"; -- CELLO
            WHEN "11" => output <= "110001101101011"; -- FEPPO
        END CASE;
    END PROCESS;
END Behavior;
```

Figure 5. The multiplexer code, selecting different words (different group of 3-bits corresponding to different letter groups)

```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.ALL;
3
4   ENTITY shifter IS
5       PORT (
6           input : IN STD_LOGIC_VECTOR(14 DOWNTO 0);
7           sel: IN STD_LOGIC_VECTOR(2 DOWNTO 0);
8           output : OUT STD_LOGIC_VECTOR(14 DOWNTO 0)
9       );
10  END shifter;
11
12  ARCHITECTURE Behavior OF shifter IS
13  BEGIN
14      PROCESS (input, sel)
15      BEGIN
16          CASE sel IS
17              WHEN "000" => output <= input;
18              WHEN "001" => output <= input(11 DOWNTO 0) & input(14 DOWNTO 12);
19              WHEN "010" => output <= input(8 DOWNTO 0) & input(14 DOWNTO 9);
20              WHEN "011" => output <= input(5 DOWNTO 0) & input(14 DOWNTO 6);
21              WHEN "100" => output <= input(2 DOWNTO 0) & input(14 DOWNTO 3);
22              WHEN OTHERS => output <= input;
23          END CASE;
24      END PROCESS;
25  END Behavior;
26
```

Figure 6. the code of the shifter module that takes a 15-bit input vector and shifts its contents based on a 3-bit selection (sel). If no valid shift is selected, the input remains unchanged.

The architecture of the decoder7 hasn't changed much since the last part except minor changes to accommodate for the larger number of characters to be displayed:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY decoder7 IS
    PORT (
        C : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
        Display : OUT STD_LOGIC_VECTOR(0 to 6)
    );
END decoder7;

ARCHITECTURE Behavior OF decoder7 IS
BEGIN
    PROCESS (C)
    BEGIN
        Display <= "1111111"; -- Reducing undefined errors in the Time simulation
        IF C = "000" THEN Display <= "1001000"; -- Display 'H'
        ELSIF C = "001" THEN Display <= "0110000"; -- Display 'E'
        ELSIF C = "010" THEN Display <= "1110001"; -- Display 'L'
        ELSIF C = "011" THEN Display <= "0000001"; -- Display 'O'
        ELSIF C = "100" THEN Display <= "0110001"; -- Display 'C'
        ELSIF C = "101" THEN Display <= "0011000"; -- Display 'P'
        ELSIF C = "110" THEN Display <= "0111000"; -- Display 'F'
        ELSE Display <= "1111111"; -- Default blank
        END IF;
    END PROCESS;
END Behavior;
```

Figure 7. The code of the new decoder, covering also the letters P, F, C.

The **Part2 module** serves as the top-level entity, having as of components the multiplexer named mux**,** shifter**,** and the decoder7. It was noted during lab testing that the switches were connected in reverse order. To quickly resolve this issue, the range of the Display vector in the VHDL code was changed from (0 downto 6) to (0 to 6). This was a quicker trouble shooting compare to changing the pin assignments.

```
ENTITY Part2 IS

PORT ( SW : IN STD_LOGIC_VECTOR(4 DOWNTO 0);
HEX0,HEX1,HEX2,HEX3,HEX4 : OUT STD_LOGIC_VECTOR(0 to 6));
END Part2;

ARCHITECTURE Behavior OF Part2 IS
component mux IS
PORT ( sel: IN STD_LOGIC_VECTOR(1 downto 0);
output : OUT STD_LOGIC_VECTOR(14 downto 0));
end component;


component shifter IS
PORT ( input : IN STD_LOGIC_VECTOR(14 downto 0);
sel: IN STD_LOGIC_VECTOR(2 downto 0);
output : OUT STD_LOGIC_VECTOR(14 downto 0));
end component ;


COMPONENT decoder7
PORT ( C : IN STD_LOGIC_VECTOR(2 downto 0);
Display : OUT STD_LOGIC_VECTOR(0 to 6));
END COMPONENT;

SIGNAL a1,a2 : std_logic_vector (14 downto 0);


BEGIN
MUX0: mux PORT MAP (SW(1 DOWNTO 0),a1);
SHIFT0: shifter PORT MAP (a1, SW(4 downto 2),a2);
H0: decoder7 PORT MAP (a2(2 downto 0), HEX0);
H1: decoder7 PORT MAP (a2(5 downto 3), HEX1);
H2: decoder7 PORT MAP (a2(8 downto 6), HEX2);
H3: decoder7 PORT MAP (a2(11 downto 9), HEX3);
H4: decoder7 PORT MAP (a2(14 downto 12), HEX4);
END Behavior;
```

Figure 8. the top-level code integrating all the components, with port mapping all the input and output signals.

The testbench for functional simulation is defined in **"tb_part2.vhd"**. The input signal **C** is varied, and signals defined in test bench are mapped to the device ports. A 10 ns delay between each change of input C, ensures stable values for observation. Below is the figures of the VHDL code of the test bench and the timing simulation waveform results.

```
46
47      -- Test case 1: All zeros, meaning the result should be "HELLO", "1001000"
48      SW <= "00000";
49      WAIT for 10 ns;
50
51      -- Test case 2: shifting Hello to "ELLOH"
52      SW <= "00001";
53      WAIT for 10 ns;
54
55      -- Test case 3: shifting Hello to "LLOHE"
56      SW <= "00010";
57      WAIT for 10 ns;
58
59      -- Test case 4: shifting Hello to "LOHEL"
60      SW <= "00011";
61      WAIT for 10 ns;
62
63      -- Test case 5: shifting Hello to "OHELL"
64      SW <= "00100";
65      WAIT for 10 ns;
66
67      -- Test case 6: choosing the word CEPPO shifting to "PPOCE"
68      SW <= "01010";
69      WAIT for 10 ns;
70
71      -- Test case 7: choosing the word CELLO, no shifting as 101 is treated like the defult based on the truth table.
72      SW <= "10101";
73      WAIT for 10 ns;
74
75      -- Test case 8: choosing the word FEPPO, no shifting as 111 is treated like the defult based on the truth table.
76      SW <= "11111";
77      WAIT for 10 ns;
78
79
80      WAIT;  -- Wait forever - simulation ends here.
81    END PROCESS;
82
83  END behavior;
```

Figure 9. the code of the test bench for the part two, shifting and choosing the words to be displayed in five HEX's (seven segment display input 7-bit vectors).

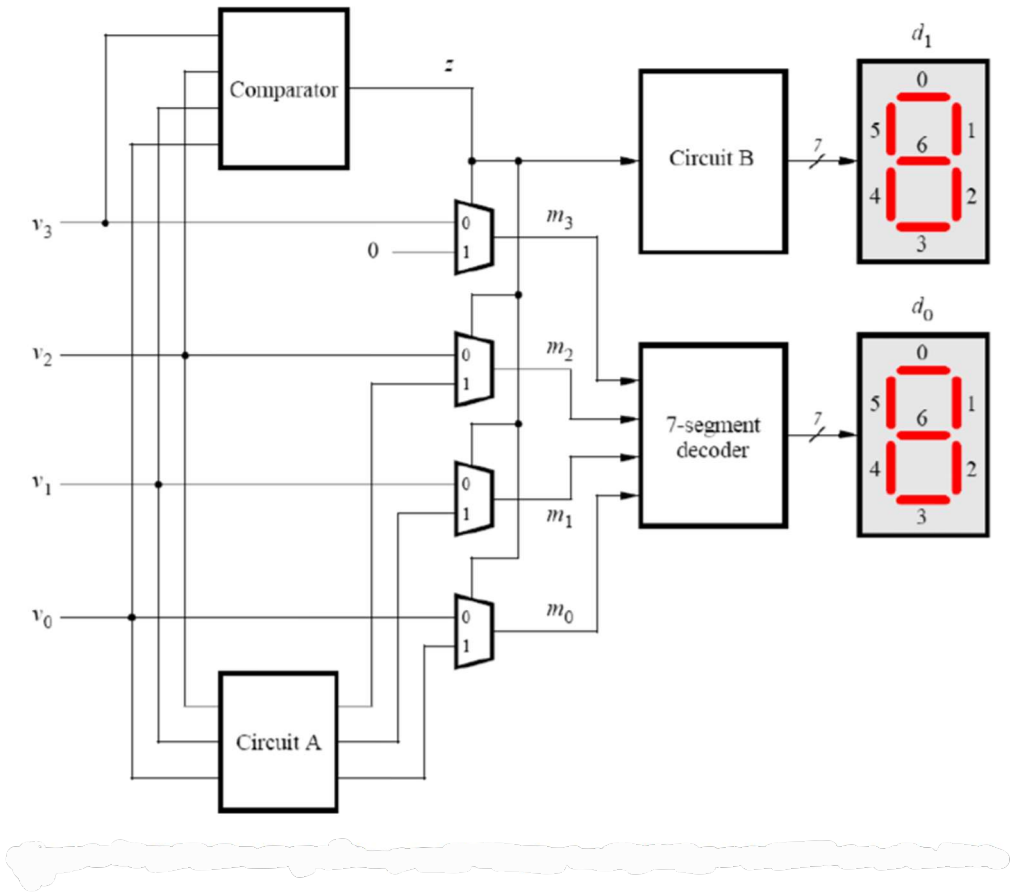
Figure 10. test bench simulation results for part two, the red lines indicate undefined values for the signals at the start of the timing simulation. and the changes was carried out with delays.

3. Binary to decimal converter

The third part focuses on designing a binary-to-decimal converter, named “bin2dc”, which converts a 4-bit binary number (V = v_3 v_2 v_1 v_0) (input via switches (SW_3-0)) into its two-digit decimal equivalent, displayed on HEX1 and HEX0. The design is implemented in VHDL using a hierarchical structure, with the main entity bin2dc defined in “bin2dc.vhd”. The converter consists of four 1-bit 2-to-1 multiplexers (mux.vhd), a comparator (comparator.vhd), “Circuit A” (circuitA.vhd). The structural connections between the components are defined in bin2dc.vhd, ensuring proper mapping from logic to display outputs. All. The design follows a **modular structure**, implementing the conversion using a **comparator, multiplexers, and a decoder**. The final output is dynamically displayed based on the binary input.

Binary value	Decimal digits
0000	0 0
0001	0 1
0010	0 2
1001	0 9
1010	1 0
1011	1 1
1100	1 2
1101	1 3
1110	1 4
1111	1 5

Table 4: Binary-to-Decimal conversion.



Starting with the comparator, here we determine whether the binary number is greater than 9 (1001). Then it outputs z = 1 if the number is **greater than 9**, indicating that conversion is required. If z = 0, it means that the number was equal or below 9, so the binary input should remain unchanged. We used ieee.std_logic_unsigned.all library in the comparator to enable direct binary comparisons (>) on std_logic_vector without manual data type conversion. Here in Figure 11 is the VHDL code of the comparator module.

```
12  architecture behave of comparator is
13  begin
14    process (v)
15    begin
16      -- If v > "1001" (decimal 9), then z='1'
17      if v > "1001" then
18        z <= '1';
19      else
20        z <= '0';
21      end if;
22    end process;
23
24  end behave;
25
```

Figure 11. the architecture used for comparator module.

Next, the **circuit A** component is for if the number is greater than 9, subtracts 2 (010) to adjust the higher digit for decimal representation:

```
architecture behave of circuitA is
begin
   -- Subtract 2 "010" from the three bits
   m <= v - "010";
end behave;
```

Figure 12. the architecture used for circuitA module.

Circuit B is like a decoder only for the tens digit, thus responsible for showing the tens digit (HEX1) of our decimal number. It takes a 1-bit input (z) from the **comparator** and determines whether the tens digit should display 1 or remain blank (0). Below is shown the VHDL code of the architecture of this component :

```
11  architecture behave of circuitB is
12  begin
13    process (z)
14    begin
15      if (z = '1') then
16        d1 <= "1111001";
17      else
18        d1 <= "0000001";
19      end if;
20    end process;
21  end behave;
22
```

Figure 13. the architecture used for circuitB module.

Finally, in the **bin2dec** module, we connect all components and the input and output signals together with correct port mapping to enable binary-to-decimal conversion. In summary, the **comparator** checks if the input exceeds 9, activating **Circuit A** to adjust the value and **Circuit B** to show the **tens digit** (HEX1).

For the **multiplexers,** they select the correct binary output for the four bits, either the exact bits equal to the input ones , or the given bits from the circuit A. This selection is based on the value of the z signal, which is given from comparator module. The outputs of the three multiplexers are the inputs for the **7-segment decoder** which displays the ones digit on HEX0. This integration ensures accurate and efficient conversion.

```
begin

  u_comparator: comparator port map (v => v, z => z);

  u_circuitA: circuitA port map (v => v(2 downto 0), m => n);

  u_circuitB: circuitB port map (z => z, d1 => HEX1);

  dec: decoder7 port map (C => m, Display => HEX0);

  M3: mux port map (A => v(3), B => '0', sel => z, M => m(3));
  M2: mux port map (A => v(2), B => n(2), sel => z, M => m(2));
  M1: mux port map (A => v(1), B => n(1), sel => z, M => m(1));
  M0: mux port map (A => v(0), B => n(0), sel => z, M => m(0));

end structural;
```

Figure 14. after defining all the components, mapping corresponding signals based on the circuit structure of the binary to decimal circuit.

To verify the functionality of the bin2dec module, a testbench (part3_tb) was implemented. It sequentially applies different 4-bit binary inputs (v), waiting 100 ns between each test case, and observes the corresponding 7-segment display outputs (HEX1 and HEX0). The simulation ensures correct BCD conversion for values 0 to 15. Choosing 100ns is because the timing simulation has initial delays (red lines are undefined values for the output signals) and delays after changings of the inputs, all of which are in magnitude of 10's nano seconds. Therefore using 100 ns waiting time before changing the input signal is more reasonable and gives a better outcome, shown in figure 16.

```
19
20  begin
21      uut: bin2dec port map (
22           v    => v,
23           HEX0 => HEX0,
24           HEX1 => HEX1
25      );
26
27      process
28      begin
29           v <= "0000";
30           wait for 100 ns;
31           v <= "0001";
32           wait for 100 ns;
33           v <= "0010";
34           wait for 100 ns;
35           v <= "0011";
36           wait for 100 ns;
37           v <= "0100";
38           wait for 100 ns;
39           v <= "0111";
40           wait for 100 ns;
41           v <= "1011";
42           wait for 100 ns;
43           v <= "1111";
44           wait;
45      end process;
46  end behavior;
47
48
```

Figure 15. the VHDL code for test bench that simulates the binary to decimal circuit named as bin2dec.

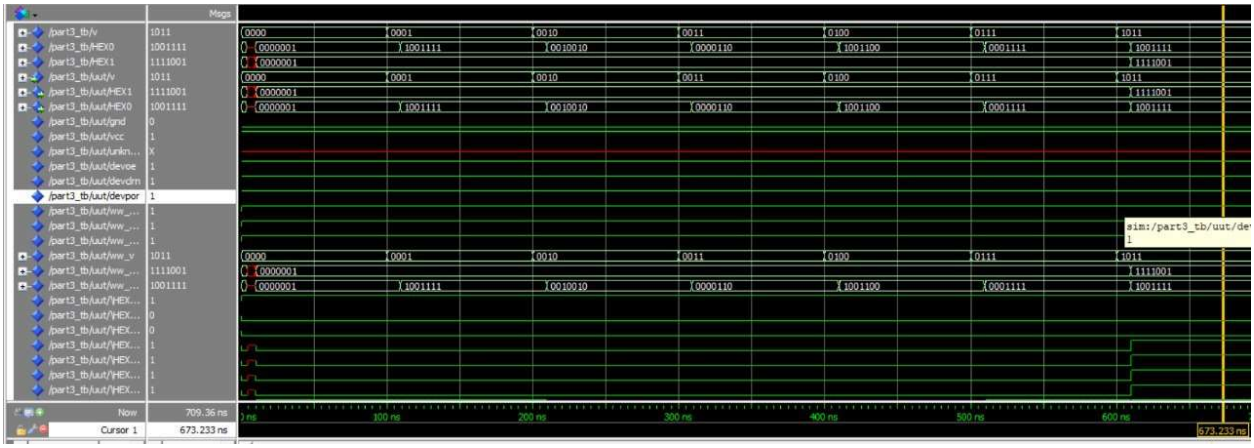
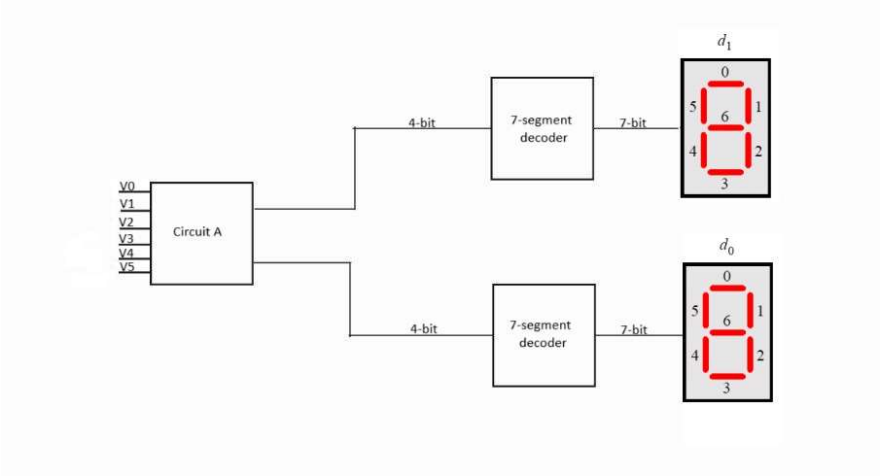


Figure 16. timing simulation waveform results of the binary to decimal circuit.

4. Binary to BCD converter



The objective of this section is to design a **combinational circuit** that converts a **6-bit binary number** (0–63) into **Binary-Coded Decimal (BCD)** format. The converted BCD digits are displayed on **two 7-segment displays**, where one represents the **tens** digit and the other represents the **ones** digit. The design follows a **modular approach**, reusing the **decoder7** from the previous part. The conversion process is achieved using a new **circuitA ,** different from the previous circuit A used in previous part, which determines the **tens** and **ones** digits through **conditional iterative subtraction**. The condition is that if the temp is more or equal to 10, it is subtracted by 10. There is also a counter variable named t, that counts the tens digit value on each iteration. The final output signals (tens, ones) are sent to **decoder7** modules, driving the corresponding **HEX displays**.

```
20    begin
21      temp := v;
22      t := (others => '0');  -- Initialize tens to 0
23      o := temp(3 downto 0); -- Initialize ones as lower 4 bits
24
25      -- Subtract 10 repeatedly to get the tens digit
26      if temp >= "001010" then t := t + "0001"; temp := temp - "001010"; end if; -- 10
27      if temp >= "001010" then t := t + "0001"; temp := temp - "001010"; end if; -- 20
28      if temp >= "001010" then t := t + "0001"; temp := temp - "001010"; end if; -- 30
29      if temp >= "001010" then t := t + "0001"; temp := temp - "001010"; end if; -- 40
30      if temp >= "001010" then t := t + "0001"; temp := temp - "001010"; end if; -- 50
31      if temp >= "001010" then t := t + "0001"; temp := temp - "001010"; end if; -- 60
32
33      -- Remaining value is the ones place
34      o := temp(3 downto 0);
35
36      tens <= t;
37      ones <= o;
38    end process;
39  end behave;
40
```

Figure 17. the architecture used for circuitA module for binary to BCD, the iterative subtraction happens when **temp** is more than 10, and the counter **t** is incremented by one. Then variable **t** is stored in output 4-bit signal **tens**. And the final value of the temp (which is definitely smaller than 10) is stored at the output 4-bit signal **ones**.

In summary, the **bin2bcd** module integrates components : **circuitA** – Converts binary to ones and tens (BCD digits), **decoder7** – Maps BCD digits to **7-segment display** outputs. In Figure 20, the VHDL code of the bin2bcd is given.

```
architecture structural of bin2bcd is

  -- Internal signals
  signal m0, m1 : std_logic_vector(3 downto 0);

  -- Components
  component circuitA is
    port (
      v : in std_logic_vector(5 downto 0);
      tens : out std_logic_vector(3 downto 0);
      ones : out std_logic_vector(3 downto 0)
    );
  end component;

  component decoder7 is
    port (
      C       : in std_logic_vector(3 downto 0);
      Display : out std_logic_vector(0 to 6)
    );
  end component;

begin
  -- Convert binary to BCD (Tens and Ones)
  u_circuitA: circuitA port map (v => v, tens => m0, ones => m1);

  -- Display the digits on 7-segment displays
  dec1: decoder7 port map (C => m0, Display => HEX1);
  dec0: decoder7 port map (C => m1, Display => HEX0);

end structural;
```

Figure 20. the architecture of bin2bcd, defining the two components : circuitA and decoder7, and mapping the input signals and output signals according to the circuit structure of the bin2bcd.

A testbench was implemented to verify the bin2bcd functionality, running multiple test cases with a 100 ns delay between each input:

```
BEGIN

    -- Instantiate the binary-to-BCD converter
    uut: bin2bcd PORT MAP (v => v, HEX1 => HEX1, HEX0 => HEX0);

    -- Stimulus process
    PROCESS
    BEGIN
        -- Test Case 1: Binary 0 (000000) -> BCD (0, 0)
        v <= "000000"; WAIT FOR 100 ns;

        -- Test Case 2: Binary 5 (000101) -> BCD (0, 5)
        v <= "000101"; WAIT FOR 100 ns;

        -- Test Case 3: Binary 10 (001010) -> BCD (1, 0)
        v <= "001010"; WAIT FOR 100 ns;

        -- Test Case 4: Binary 15 (001111) -> BCD (1, 5)
        v <= "001111"; WAIT FOR 100 ns;

        -- Test Case 5: Binary 19 (010011) -> BCD (1, 9)
        v <= "010011"; WAIT FOR 100 ns;

        -- Test Case 6: Binary 20 (010100) -> BCD (2, 0)
        v <= "010100"; WAIT FOR 100 ns;

        -- Test Case 7: Binary 35 (100011) -> BCD (3, 5)
        v <= "100011"; WAIT FOR 100 ns;

        -- Test Case 8: Binary 51 (110011) -> BCD (5, 1)
        v <= "110011"; WAIT FOR 100 ns;

        -- End simulation
        WAIT;
    END PROCESS;
```

100ns was used as of changing time delay because after time simulation it has been seen that the delays and hazards were interfering with the outputs .however, with 100ns changing time delay the results were more reasonable and standard, still with some hazards (red lines) and delays, all of which can be seen in below screenshot of the waveform. The results of the timing simulation for bin2bcd is shown in figure 21.

Figure 21. timing simulation waveform results of the binary to BCD circuit.

Conclusion

In conclusion, every project was successfully tested in the lab, with a few difficulties during the process, such as correcting the reversed pin assignments, by changing the VHDL code instead. Or figuring the 100ns instead of 10ns for timing simulation, in order to get proper results. All of which were resolved efficiently through teamwork.